COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/14/26	**DATE SUBMITTED:**	10/14/26
NAME:	Arididdon, Gabriel Dwyane D. Esguerra, Paula G.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO **Hands-on Activity 12.1 Algorithmic Strategies**

1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

Table 12-1.

Algorithmic Strategies and Examples

Table 12-1. Algorithmic Strategies and Examples

Strategy	Algorithm	Analysis
Recursion	Fibonacci Sequence	This uses recursion because each number is defined directly as the sum of the two preceding numbers
Brute Force	Dictionary Attack	It tries every possible solution until it matches a correct answer. An example for this is password cracking.
Backtracking	Depth First Search	The idea of depth first search is where it starts from the given source then explores the one path as deeply as possible, it might start from the left then to its branches, then moves on to the right.
Greedy	Dijkstra's algorithm	It finds the shortest path distance from the source vertex to all the other

		vertices in the graph/code.
Divide-and-Conquer	Quick Sort or Merge Sort	In merge sort, it first divides the array until it sorts them one by one then merges or conquers the array. Merge sort is the same as quick sort, the difference is quick sort uses pivot, it all starts with pivot, merges then conquers.

Table 12-2.

Memoization Implementation

```cpp
#include <iostream>
#include <algorithm>
#include <vector>


const int MAXN = 1000;
std::vector<int> memo(MAXN, -1); // Initialize memo array of size 1000 with -1

int getMinSteps(int n) {
    if (n == 1)
        return 0; // Base case: 1 requires 0 steps

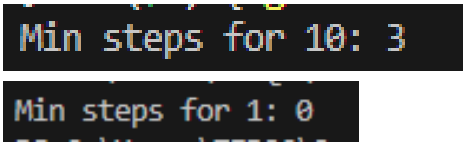
    // 1. Look up in cache
    if (memo[n] != -1)
        return memo[n];

    // 2. Compute if not in cache
    int r = 1 + getMinSteps(n - 1);
    if (n % 2 == 0) r = std::min(r, 1 + getMinSteps(n / 2));
    if (n % 3 == 0) r = std::min(r, 1 + getMinSteps(n / 3));

    // 3. Store in cache before returning
    memo[n] = r;
    return r;
}

int main() {
    int n = 10;
     std::cout << "Min steps for " << n << ": " << getMinSteps(n) <<
```

```
std::endl;
    return 0;
}
```

Screenshot


Aridon	
Analysis	It matches the optimal way which is 3 steps: (10-1) = 9 9/3 = 3 3/3 = 1 It first subtracts a number (n) by 1 then checks if the answer is divisible by 2 or 3 then the program stops and counts the number of steps if it reaches a value of 1.

Esguerra	
Analysis	It checks the n number, if it is not divisible by 3 or 2, it subtracts by 1, then repeat the code, if it is now divisible by 2 or 3, it now divides until it gets to number if one, now it repeats the code, now the number is now at 1, it exits.

Table 12-3.

Bottom-Up Dynamic Programming Implementation

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

int getMinSteps(int n) {
    if (n <= 1) return 0;

    std::vector<int> dp(n + 1);
    dp[1] = 0;

    for(int i = 2; i <= n; i++) {
        dp[i] = 1 + dp[i-1];
        if (i % 2 == 0) dp[i] = std::min(dp[i], 1 + dp[i/2]);
        if (i % 3 == 0) dp[i] = std::min(dp[i], 1 + dp[i/3]);
    }
    return dp[n];
}

int main() {
    int n = 10;
      std::cout << "Min steps for " << n << ": " << getMinSteps(n) <<
std::endl;
    return 0;
}
```

Screenshot	
	
Aridido n	
Analysis	It has the same output as the memoization approach. The difference between them is that memoization starts from the to or target N, until it world down to base case 1. On the other, hand this approach (bottom-up) starts from the base case 1 and works up to target N using iteration and it solves all possible subproblems sequentially.
Esguerra	

Analysis	It has the same implementation of memoization, the difference is Bottom-up uses iteration and for loop, it solves the trivial subproblem then to the given problem to get the answer.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

LO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Pseudocode (+ Solving Problem by Hand)

The path for 25 is 0
The first path is 4 + 7 + 1 + 3 + 1 + 2 and the destination cell 3 which is equal to 21
For path 26, there are 2 paths. The first is 4 + 7 + 1 + 3 + 1 + 7 + 3 then another is 4 + 6 + 7 + 3 + 1 + 2 + 3

Working C++ Code

```cpp
#include <iostream>
#include <vector>
#include <map>

// Recursive function with memoization
int countPaths(int i, int j, int remaining_cost, const std::vector<std::vector<int>>& mat,
               std::vector<std::vector<std::map<int, int>>>& memo) {
    int M = mat.size();
    int N = mat[0].size();

    // Base Case 1: Out of bounds or cost dropped below zero
    if (i >= M || j >= N || remaining_cost < 0) {
        return 0;
```

```
    }

    // Base Case 2: Reached the destination cell
    if (i == M - 1 && j == N - 1) {
        return (remaining_cost == mat[i][j]) ? 1 : 0;
    }

    // Check if this state has already been calculated
    if (memo[i][j].count(remaining_cost)) {
        return memo[i][j][remaining_cost];
    }

    // Move right and move down, deducting current cell's cost
    int next_cost = remaining_cost - mat[i][j];
    int right_paths = countPaths(i, j + 1, next_cost, mat, memo);
    int down_paths = countPaths(i + 1, j, next_cost, mat, memo);

    // Save and return the total paths for this state
    return memo[i][j][remaining_cost] = right_paths + down_paths;
}

int main() {
    // Example 4x4 matrix from the problem statement
    std::vector<std::vector<int>> matrix = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int target_cost = 25;
    int M = matrix.size();
    int N = matrix[0].size();

    // 3D Memoization structure: memo[row][col][remaining_cost]
      // Using a map for the 3rd dimension avoids allocating excessive memory for large
costs
         std::vector<std::vector<std::map<int,  int>>>  memo(M,  std::vector<std::map<int,
int>>(N));

    int total_paths = countPaths(0, 0, target_cost, matrix, memo);
```

```
    std::cout << "Number of paths with cost " << target_cost << " is: " << total_paths <<
std::endl;

    return 0;
}
```

Analysis of working code

Arididdon	Esguerra
The code uses a top down or memoization approach with recursion since it strictly moves right and down to reach the target cell (M-1, N-1). Each iteration, the cell value is deducted from the remaining_cost while it checks if it is already calculated before so it doesn't repeat it again or overlap with other subproblems and have a memory overflow.	This code uses memoization, where it uses recursion. It starts with finding the number of 25, we give the number given in the procedure to check every possible iteration if there is an output of 25, whereas there is no output of 25 in all combinations. It only shows the 21, 26 and above. (i, j) are the row and column respectively, the if condition is to check the limits of the matrix, where if the code checks outside the matrix, it will give an error. The check goes as moving right (j +1) and moving down(i+1), this will serve as the alternating check of every path as possible, then the total paths are saved so it will not repeat the previous calculated array.

Screenshots of demonstration

```
Number of paths with cost 25 is: 0
Number of paths with cost 26 is: 2
Number of paths with cost 21 is: 1
```

Conclusion

Arididdon	Esguerra
In this activity, I learned and recalled different algorithmic strategies and how top-down (memoization) and bottom-up works in getting the base case with fewer steps using recursion or iteration. Memoization uses recursion and starts from the top or N then works down until it reaches base case 1. On the other hand, bottom-up starts from the base case 1 and works up to the target value N. Either way, both works the same it's just that they have different approaches or direction.	Algorithmic Strategies are approaches to how to solve a problem, it explores every possible way. The approaches are recursion, brute force, backtracking , divide and conquer and dynamic programming. Now in this activity, we used recursion, where if it can be first checked if it is divisible, if not, it should be subtracted by one and repeat, check if it is now divisible, then if yes, divide to the given number in the code. It is also possible for the code to find in all possible combinations to find a specific total. And for the case in conclusion, we are to find the total number of 25, where it is not found, but it found all possible iterations after 26. I have learned the possible algorithmic strategies and maybe planning to use them in the future, in multi purposes.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explaining concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **We used Gemini to explain hard concepts and also with some code assistance in supplementary activity**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.